

PROMPT - 1

Read the uploaded Backend_FINAL document carefully.

The document is the source of truth for architecture and business logic.

IMPORTANT:

Do NOT implement the entire document at once.

Your goal is to build a production-quality MVP that can generate a valid ranked candidate list from the provided dataset.

Follow these rules:

1. Implement only the minimum modules required for a working end-to-end ranking system.
2. Ignore advanced modules for now:
 - Analytics
 - Dashboard metrics
 - Advanced behavioral modeling
 - Complex honeypot sub-engines
 - Explainability visualizations
 - Multi-stage optimization
3. Build the backend in this exact order:

Phase A:

- config.py
- data_loader.py
- jd_parser.py

Requirements:

- Stream candidates.jsonl efficiently
- Parse job_description.docx
- Extract hard skills
- Extract preferred skills
- Extract experience requirements
- Create a normalized requirements object

Phase B:

- embedding_engine.py

Requirements:

- Use sentence-transformers all-MiniLM-L6-v2
- Generate JD embeddings
- Generate candidate embeddings
- Support pre-computation and caching

Phase C:

- title_filter.py

Requirements:

- Remove obviously irrelevant candidates
- Keep AI/ML/Data candidates
- Maintain high recall
- Log filter statistics

Phase D:

- skill_scorer.py
- experience_scorer.py
- career_analyzer.py

Requirements:

- Produce normalized scores between 0 and 1
- Avoid hard-coded company bias
- Prefer evidence-based scoring from career history

Phase E:

- scoring_engine.py

Composite score:

```
final_score =
0.45 * skill_score +
0.30 * career_score +
0.15 * experience_score +
0.10 * embedding_similarity
```

Normalize all outputs.

Phase F:

- ranker.py

Requirements:

- Score all candidates
- Sort descending

- Generate Top 100

Phase G:

- reasoning_generator.py

Requirements:

- Generate deterministic reasoning
- Use actual profile data only
- No hallucinations
- Mention skills, experience, and career evidence

Phase H:

- output_writer.py

Requirements:

- Produce submission.csv
- Follow competition format exactly

Phase I:

- rank.py

Requirements:

- Single command execution

Example:

```
python rank.py
--candidates candidates.jsonl
--jd job_description.docx
--output submission.csv
```

Architecture Requirements:

- Python 3.11+
- Fast execution
- CPU compatible
- No external APIs
- No Gemini
- No OpenAI
- No network dependencies

Code Quality Requirements:

- Modular architecture

- Type hints
- Logging
- Error handling
- Clean folder structure
- Production-ready code

Deliverables:

1. Folder structure
2. File-by-file implementation plan
3. Dependencies list
4. MVP implementation
5. Step-by-step testing instructions

Build only the MVP necessary to produce a valid submission.csv.
Do not implement optional features until MVP is complete and tested.

PROMT – 2 (AFTER MVP WORKS)

The MVP ranking system is working.

Now implement Phase 2 enhancements:

1. Honeypot Detection
2. Behavioral Scoring
3. Candidate Validation
4. Advanced Feature Caching
5. Cross-Encoder Re-Ranking

Requirements:

Pipeline:

```

Candidate
↓
Embedding Search
↓
Top 1000
↓
Feature Scoring
↓

```

Top 300



Cross Encoder Re-Ranking



Top 100



CSV

Use:

cross-encoder/ms-marco-MiniLM-L-6-v2

Add performance optimizations and caching.

Do not modify existing MVP interfaces.

Extend the architecture cleanly.